

Technical Security Assessment Report

Analysis of Python Library Hijacking Vulnerabilities

Vulnerability Type: Privilege Escalation / Path Traversal / Environment Manipulation

Target System: Linux Linux-based Developer Workstations / Servers

Severity: **High (under sudo / SUID contexts)**

1. Executive Summary

Python library hijacking is an execution manipulation vector that occurs when an attacker exploits system misconfigurations to alter or redirect the third-party dependencies loaded by an application. If a highly privileged process (such as a root-level sudo script or system service) executes a Python application under these conditions, the underlying operating system may fetch a malicious artifact instead of the legitimate package. This report explores the core operational reasons behind this risk and summarizes the specific Tactics, Techniques, and Procedures (TTPs) involved.

2. Why Python is Susceptible (Root Cause Analysis)

Python determines dependency location by resolving sequential path orders (defined under `sys.path`). The search hierarchy naturally evaluates local directory scopes and environment configurations prior to looking into globally restricted directories. Consequently, risks materialize when:

- **Implicit Path Priorities:** Directories configured early in the module lookup array override the standardized site-packages installation folders.
- **Loose File Permissions:** Broad write permissions (e.g., 777 or global write bits) allow less privileged execution contexts to modify core functions within root-owned systems.
- **Environment Inheritance:** Elevated execution vectors like sudo when configured with flags like `SETENV` allow normal users to define custom lookup arrays via variables like `PYTHONPATH`.

3. Tactics, Techniques, and Procedures (TTP Analysis)

TTP 1: Insecure Write Permissions on Existing Libraries

When an existing legitimate Python package is globally writable, an attacker can modify its inner functional blocks directly. If an administrative task subsequently executes a script depending on that function, the malicious code runs inline under the parent privilege.

Mechanism: Modifying pre-existing system modules.

Example Path Impacted: /usr/local/lib/python3.8/dist-packages/psutil/__init__.py

```
# Injecting commands directly into valid system packages
def virtual_memory():
    import os
    os.system('id') # Executed with caller permissions
    ...
```

TTP 2: Search Path Order Manipulation (Python Search Path)

Python traverses the `sys.path` listing linearly. If an attacker discovers that an earlier, high-priority directory on the search path contains broad write configuration permissions, they can plant a malicious file named identically to the legitimate target module (e.g., `psutil.py`).

```
# User uncovers a globally write-permissive system search path
$ ls -la /usr/lib/python3.8

# Creating an overlapping file inside the preferred directory order
# File: /usr/lib/python3.8/psutil.py
import os
def virtual_memory():
    os.system('id')
```

TTP 3: Environment Variable Hijacking (PYTHONPATH)

If the administrative permission configuration layer allows the injection or modification of execution environments (e.g., `SETENV`: explicitly allowed in `/etc/sudoers`), users can prepend arbitrary paths to the lookup logic at runtime.

```
$ sudo PYTHONPATH=/tmp/ /usr/bin/python3 ./mem_status.py
```

This instructs the python interpreter to seamlessly parse `/tmp/` first, prioritizing any mock script variants located inside that directory.

4. Mitigation & Remediation Guidelines

- **Enforce Strict File Permissions:** Audit all folders included inside python system environments. Ensure standard permissions limit write access solely to administrative or root accounts (e.g., `chmod 755`).
- **Restrict Sudoers Configurations:** Avoid assigning blanket `SETENV` permissions within the `/etc/sudoers` layout. Ensure environmental isolation mechanisms like `env_reset` remain active.

- **Use Secure Pathing:** Invoke Python explicitly via exact system paths and avoid working with untrusted local folder mappings within mission-critical operational processes.